Programación declarativa @ URJC

Programación funcional

Curso 25-26, convocatoria extraordinaria (junio de 2026)

Campus de Móstoles/Vicálvaro

La duración del examen es de 1h:20m.

Preámbulo

In [ ]:
```
import $ivy.`org.scalatest::scalatest:3.2.16`
import org.scalatest.{Filter => _, _}, flatspec._, matchers._
```

Algunas definiciones de tipos y funciones auxiliares

In [ ]:
```
object Signatures:
    abstract class List[A]:

        // Common HOFs
        def foldRight[B](nil: B)(cons: (A, B) => B): B
        def foldLeft[B](initial: B)(update: (B, A) => B): B
        def map[B](f: A => B): List[B]
        def flatMap[B](f: A => List[B]): List[B]
        def filter(f: A => Boolean): List[A]
        def forall(pred: A => Boolean): Boolean
        def exists(pred: A => Boolean): Boolean

        // Common functions
        def length: Int
        def reverse: List[A]
        def drop(i: Int): List[A]
        def appended(e: A): List[A]

        // Empareja cada elemento con su índice (empezando en 0)
        def zipWithIndex: List[(A, Int)]

        // Suma los elementos de una lista numérica
        def sum: A
```

In [ ]:
```
// Potencia de enteros: pow(base, exp) = base elevado a exp
def pow(base: Int, exp: Int): Int =
```

```
    math.pow(base, exp).toInt
```

In [ ]:
```
def divideAndConquer[A, B](l: List[A]): B =
    l match
        case Nil => ???
        case head :: tail =>
            val tailSol: B = divideAndConquer(tail)
            ???(head, tailSol)
```

In [ ]:
```
def iterative_tailrecursion[A, B](l: List[A]): B =

    def step(out: B, aux: List[A]): B =
        aux match
            case Nil => out
            case head :: tail =>
                step(???(out, head), tail)

    step(???, l)
```

Definiciones auxiliares sobre la correspondencia Curry-Howard y Tipos Algebraicos de Datos

	Function methods	Function values
Function1	`def f(a: A): B = ??? : B`	`val f: A => B = (a: A) => ??? : B`
Function2	`def f(a: A, b: B): C = ??? : C`	`val f: (A, B) => C = (a: A, b: B) => ??? : C`
Currified functions	`def f(a: A)(b: B): C = ??? : C`	`val f: A => B => C = (a: A) => (b: B) => ??? : C`
Generic functions	`def f[A, B](a: A): B = ??? : B`	`val f: [A, B] => A => B = [A, B] => (a: A) => ??? : B`

In [ ]:
```
trait Isomorphic[A, B]:

    def from(a: A): B

    def to(b: B): A

    // equality

    def equalA(a1: A, a2: A): Boolean =
        a1 == a2

    def equalB(b1: B, b2: B): Boolean =
        b1 == b2

    // Bijection laws

    def law1(a: A): Boolean =
        equalA(to(from(a)), a)
```

```
    def law2(b: B): Boolean =
        equalB(from(to(b)), b)
```

Type-driven development rules	Constructor	Destructor
(*exponents*) A => B	((a: A) => ??? : B): **(A => B)**	val f: **A => B** = … val a: A = … f(a): **B**
(*products*) (A, B)	(??? : A, ??? : B): **(A, B)**	val t: **(A, B)** = … t._1 : **A** t._2 : **B**
(1) Unit	(): Unit	
(*sums*) Either[A, B]	Left(??? : A): **Either[A, B]** Right(??? : B): **Either[A, B]**	val e: **Either[A, B]** = … (e match { case Left(a: A) => … : **C** case Right(b: B) => … : **C** }): **C**
(0) Nothing		val n: **Nothing** = … n: **C**
(var) A		val a: **A** = … a : **A**

In [ ]:
```
type Not[P] = P => Nothing
type <=>[P, Q] = (P => Q, Q => P)
```

In [ ]:
```
trait ExcludedMiddle:
    def apply[P]: Either[P, Not[P]]
```

In [ ]:
```
trait DoubleNegation:
    def apply[P]: Not[Not[P]] => P
```

Ejercicio 1

(2 puntos)

a) (1 punto) Utiliza la correspondencia de Curry-Howard para demostrar la siguiente tautología de la lógica proposicional intuicionista:

$$\vdash (p \vee q \to r) \to (r \to s) \to (p \to s) \wedge (q \to s)$$

In [ ]:
```
// ESCRIBE TU RESPUESTA
```

b) (1 punto) Se desea utilizar la correspondencia de Curry-Howard para demostrar la validez de la siguiente tautología de la lógica clásica proposicional:

$\vdash (p \to q) \vee (q \to p)$

Para ello, utiliza como premisa adicional la ley del tercio excluso aplicada a la proposición q.

In [ ]:
```
// ESCRIBE TU RESPUESTA
```

Ejercicio 2

(2 puntos)

Demuestra que, para cualesquiera tipos `A` y `B`, los tipos de datos `Option[A] => B` y `(B, A => B)` son isomorfos, de las dos formas siguientes:

a) (0,5 puntos) Utilizando la equivalencia entre tipos algebraicos de datos y las operaciones aritméticas.

// ESCRIBE TU RESPUESTA

b) (1,5 puntos) Implementando una biyección entre ambos tipos de datos mediante una instanciación del trait `Isomorphic`.

In [ ]:
```
// ESCRIBE TU RESPUESTA
```

Ejercicio 3

(3 puntos)

Se desea implementar la función de orden superior `collect`, que recibe una función `f: A => Option[B]` y una lista de elementos de tipo `A`, y devuelve la lista de los valores `b` para los que `f` devuelve `Some(b)`, descartando los elementos en los que `f` devuelve `None` y respetando el orden original. Por ejemplo, la función deberá satisfacer los siguientes tests:

In [ ]:
```
class TestCollect(collect: (Int => Option[String], List[Int]) => List[String

    val f: Int => Option[String] =
        n => if n % 2 == 0 then Some(s"par($n)") else None

    "collect" should "work" in:
        collect(f, List(1, 2, 3, 4)) shouldBe List("par(2)", "par(4)")
        collect(f, List(1, 3, 5)) shouldBe List()
        collect(f, List()) shouldBe List()
```

a) (1 punto) Implementa la función `collect` mediante recursividad (no final).

In [ ]:
```
// IMPLEMENTA TU RESPUESTA
```

In [ ]:
```
run(TestCollect(collect))
```

b) (1 punto) Implementa la función `collect` mediante la función de orden superior `foldRight`.

In [ ]:
```
// IMPLEMENTA TU RESPUESTA
```

In [ ]:
```
run(TestCollect(collect))
```

c) (1 punto) Implementa la función `collect` mediante la función de orden superior `flatMap`.

In [ ]:
```
// IMPLEMENTA TU RESPUESTA
```

In [ ]:
```
run(TestCollect(collect))
```

Ejercicio 4

(3 puntos)

Se desea implementar la función `fromBits`, que recibe una lista de dígitos binarios (`0` y `1`), ordenados del más significativo al menos significativo, y devuelve el número decimal que representa. Por ejemplo, la lista `List(1, 0, 1, 1)` representa el número binario 1011, cuyo valor en decimal es 11. La función deberá satisfacer los siguientes tests:

In [ ]:
```
class TestFromBits(fromBits: List[Int] => Int) extends AnyFlatSpec with shou

    "fromBits" should "work" in:
        fromBits(List(1, 0, 1, 1)) shouldBe 11
        fromBits(List(1, 0, 1, 0)) shouldBe 10
        fromBits(List(1)) shouldBe 1
        fromBits(List()) shouldBe 0
        fromBits(List(1, 1, 1, 1)) shouldBe 15
```

Pista: un posible algoritmo para implementar la función `fromBits` consiste en recorrer la lista de izquierda a derecha manteniendo un acumulador `out`, inicializado a 0, que en cada paso se actualiza mediante la expresión `out * 2 + bit`, donde `bit` es el dígito procesado en ese paso.

a) (1 punto) Implementa la función `fromBits` mediante un patrón de diseño iterativo utilizando recursividad final.

In [ ]:
```
// IMPLEMENTA TU RESPUESTA
```

In [ ]:
```
run(TestFromBits(fromBits))
```

b) (1 punto) Implementa la función `fromBits` mediante la función de orden superior `foldLeft`.

In [ ]:
```
// IMPLEMENTA TU RESPUESTA
```

In [ ]:
```
run(TestFromBits(fromBits))
```

c) (1 punto) Implementa la función `fromBits` sin utilizar recursividad, mediante las funciones de orden superior `map` y `sum`. La idea consiste en invertir la lista (de modo que el bit menos significativo quede en primer lugar) y sumar cada bit multiplicado por la potencia de dos correspondiente a su posición. Es decir, si $[b_0, b_1, \ldots, b_{n-1}]$ denota la lista de bits invertida, el valor decimal viene dado por:

$$\texttt{fromBits} = \sum_{i=0}^{n-1} b_i \cdot 2^i$$

Es obligatorio implementar la función combinando exclusivamente las funciones `reverse`, `zipWithIndex`, `map`, `pow` y `sum`, sin recurrir a la recursividad ni a `foldLeft` / `foldRight`.

In [ ]:
```
// IMPLEMENTA TU RESPUESTA
```

In [ ]:
```
run(TestFromBits(fromBits))
```